RDF와 지식 그래프 쉽게 배우기

Oct 6, 2026 · @김태경 교수

Hendler 외 『Semantic Web for the Working Ontologist』 3장(RDF)을 바탕으로, 영어 원고를 만든 뒤 chatkhu(gpt-5.6-terra)로 한글화하여 쉽게 다시 정리한 학생용 자료입니다. 1~12장은 개념, 13~16장은 Python 실습이며, 실습 코드는 모두 직접 실행해 결과를 확인했습니다.

1. 큰 그림: 페이지의 웹에서 데이터의 웹으로

한 줄 요약: RDF는 "짧은 사실 카드"를 모아 두는 방법입니다.

우리가 매일 쓰는 웹(Web)은 문서의 웹입니다. 페이지는 다른 페이지로 연결됩니다. 사람은 페이지가 무엇에 관한 것인지 이해하지만, 컴퓨터는 이해하지 못합니다. Shakespeare에 관한 페이지는 Stratford에 관한 페이지로 연결될 수 있습니다. 하지만 그 연결은 둘이 어떻게 관련되는지 말해 주지 않습니다. "Shakespeare"와 "Stratford"가 현실 세계에 있는 대상이라는 것조차 말해 주지 않습니다.

시맨틱 웹(Semantic Web)은 이 문제를 해결하려고 합니다. 컴퓨터가 "Shakespeare는 사람이고, Stratford는 장소이며, 그 사람이 그 장소에 살았다"를 알게 하는 것입니다. 이런 사실을 쓰는 기본 언어가 RDF(Resource Description Framework)입니다. RDF는 시맨틱 웹의 기반입니다. RDFS와 OWL 같은 다른 언어는 RDF 위에 만들어집니다.

- 자원(resource)은 누군가 이야기하고 싶어 할 수 있는 모든 것입니다. 사람, 장소, 숫자, 심지어 "텍사스의 모든 소"도 자원입니다.
- 누구나 어떤 자원에 대해서도 지식을 추가할 수 있습니다. 이런 개방성 덕분에 일반 웹이 빠르게 성장했습니다.
- 시맨틱 웹을 만들려면 데이터가 여러 컴퓨터에 나뉘어 있어도 작동하는 저장 방법이 필요합니다. RDF가 바로 그 방법입니다.

일상 비유: RDF는 색인 카드 더미와 같습니다. 카드 한 장에는 "Shakespeare - wrote - Hamlet"처럼 짧은 사실 하나가 적혀 있습니다. 누구나 카드를 추가할 수 있고, 서로 다른 사람이 만든 카드를 같은 더미에 넣을 수 있습니다.

2. 왜 표를 낱개의 사실로 나눌까요?

한 줄 요약: 표를 "칸 하나씩" 나누면 어디서든 합칠 수 있습니다.

대부분의 데이터는 표에 저장됩니다. 각 행은 항목이고, 각 열은 속성이며, 각 칸은 값입니다. Shakespeare 시대의 작품에 관한 작은 표(ID, 제목, 저자, 매체, 연도)를 생각해 봅시다. 예를 들면 "Hamlet, Shakespeare, Play, 1604"입니다.

이 표를 여러 서버에 나누어 저장해야 한다고 해 봅시다. 방법은 세 가지입니다.

나누는 방법	각 서버가 보관하는 것	좋은 점	전체가 합의해야 하는 것
행별	전체 행	각 서버가 자기 항목에 관한 질문에 혼자 답할 수 있습니다	각 열의 의미(어느 열이 연도인가요?)
열별	전체 열	필요한 열만 쓸 수 있고, 서버를 추가해 새 열을 더할 수 있습니다	어느 행이 어느 항목인지(여기의 3행과 저기의 3행이 같은 항목인가요?)
칸별	낱개의 칸 몇 개	위 두 가지 좋은 점을 모두 가집니다	둘 다: 열의 이름과 행의 이름

RDF는 세 번째 방법인 칸별 나누기를 선택합니다. 이는 "누구나 어떤 주제에 관해서든 무엇이든 말할 수 있다(Anyone can say Anything about Any topic)"라는 구호와 맞습니다. 어떤 서버든 어떤 항목에 대해 말할 수 있고, 어떤 속성이든 쓸 수 있습니다. 대신 모든 칸에는 세 가지 정보가 필요합니다. 행의 전역 이름, 열의 전역 이름, 그리고 값 자체입니다.

3. 트리플(Triple)과 그래프(Graph)

한 줄 요약: 트리플 = 주어 + 서술어 + 목적어, 아주 짧은 문장입니다.

세 가지 정보와 함께 쓴 칸 하나를 트리플(triple)이라고 합니다. 트리플은 아주 단순한 문장처럼 읽습니다.

부분	쉬운 뜻	표에서는	예시
주어(Subject)	이야기하는 대상	행	Shakespeare
서술어(Predicate)	속성 또는 관계	열	wrote
목적어(Object)	값 또는 다른 대상	칸의 값	Macbeth

문장으로 읽으면 "Shakespeare wrote Macbeth."입니다. 트리플 예를 몇 개 더 봅시다.

Subject	Predicate	Object
Shakespeare	wrote	Macbeth
Anne Hathaway	married	Shakespeare
Shakespeare	livedIn	Stratford
Stratford	isIn	England
England	partOf	UK

여러 트리플이 같은 대상을 언급하면 그래프로 그릴 수 있습니다. 각 트리플은 주어에서 목적어로 가는 화살표가 되고, 화살표에는 서술어라는 이름표가 붙습니다. Shakespeare에 대해 아는 모든 정보가 점 하나에 모입니다. 이 점을 노드(node)라고 합니다. 표와 그래프는 같은 정보를 서로 다른 모양으로 보여 줄 뿐입니다.

데이터 병합(merge)은 쉽습니다. 출처 두 곳에 각각 트리플 더미가 있다면 두 더미를 합치면 됩니다. 두 더미에 모두 나오는 대상은 노드 하나가 됩니다. 단, 두 더미의 대상 중 어떤 것이 정말 같은 대상인지 알 수 있어야 합니다. 이것이 다음 주제입니다.

4. 대상을 구별하기: URI, 네임스페이스, 접두사

한 줄 요약: URI는 "전 세계에서 하나뿐인 주민등록번호"입니다.

"Shakespeare"라는 이름만으로는 대상을 정확히 가리킬 수 없습니다. 어디선가 누군가 영화 주인공 이름으로 쓸 수도 있습니다. "Washington"이나 "Bordeaux" 같은 이름은 더 헷갈립니다. 웹에서는 "모두가 같은 뜻으로 쓴다"고 말할 수 없습니다.

RDF는 웹이 쓰는 해결책을 빌립니다. 바로 URI(Uniform Resource Identifier)입니다. URI는 주민등록번호나 전체 우편 주소처럼 전 세계에서 하나뿐인 ID입니다. 웹 주소처럼 생겼습니다.

- URL은 페이지를 어디서 가져올지도 알려 주는 특별한 URI입니다. 모든 URL은 URI이지만, URI가 꼭 가져올 수 있는 것은 아닙니다.
- IRI는 모든 언어의 글자를 쓸 수 있는 URI입니다.
- 병합 규칙은 간단합니다. 두 노드는 URI가 같을 때, 그리고 그때에만 같은 노드입니다.

전체 URI는 길어서 두 단계로 줄여 씁니다.

- 네임스페이스(namespace)는 이름들이 공유하는 공통 앞부분입니다. 성씨처럼 생각하면 됩니다. 같은 기관에서 만든 이름들은 같은 앞부분을 씁니다. 덕분에 서로 다른 사람이 같은 단어를 다른 뜻으로 쓸 수 있습니다.
- 접두사(prefix)는 네임스페이스의 짧은 별명입니다. 접두사 `geo`를 쓰면 `England`는 `geo:England`로 씁니다. 이 줄임 표기를 CURIE(Compact URI)라고 합니다. CURIE는 줄임말일 뿐이므로, 접두사가 어떤 전체 네임스페이스를 뜻하는지 어딘가에 선언해야 합니다.

작성 규칙: URI에는 공백을 쓸 수 없어서 단어를 대문자로 이어 씁니다(CamelCase). "part of"는 `partOf`, "Measure for Measure"는 `MeasureForMeasure`가 됩니다.

W3C가 정한 표준 네임스페이스:

접두사	용도	전체 URI
rdf:	RDF의 핵심 단어 모음(작은 집합)	http://www.w3.org/1999/02/22-rdf-syntax-ns#

접두사	용도	전체 URI
rdfs:	RDF 스키마(클래스와 속성 설명)	http://www.w3.org/2000/01/rdf-schema#
skos:	용어를 정리하는 어휘	http://www.w3.org/2004/02/skos/core#
owl:	OWL, 웹 온톨로지 언어(Web Ontology Language)	http://www.w3.org/2002/07/owl#

5. 기본 제공 단어 두 개: rdf:type과 rdf:Property

한 줄 요약: `rdf:type`은 "~의 한 종류"라는 뜻입니다.

- `rdf:type`은 어떤 대상이 어떤 종류인지 말합니다. "Shakespeare rdf:type Playwright"는 "Shakespeare는 극작가이다"라는 뜻입니다. 소리 내어 읽을 때는 작은 말을 넣어 "Shakespeare [has] type Playwright"라고 읽습니다. Turtle(8장 참고)에서는 `a`가 줄임말입니다. 즉 "Shakespeare a Playwright"라고 씁니다. 유형도 유형을 가질 수 있습니다.
- `rdf:Property`는 "이 단어는 관계(서술어)로 쓰입니다"라고 알려 주는 이름표입니다. 예: "wrote rdf:type rdf:Property".

6. 스프레드시트를 RDF로 바꾸기

한 줄 요약: 행은 사물, 열은 속성, 칸은 트리플 하나가 됩니다.

ID, 모델 번호, 부서, 제품 라인, 공장 위치, SKU, 재고의 7개 열과 9개 행이 있는 제품 표를 생각해 봅시다. 이를 네 가지 간단한 규칙으로 RDF로 바꿉니다.

1. **행 하나는 사물 하나가 됩니다.** 각 행에 고유한 URI를 줍니다. 표 이름과 행 ID를 연결합니다. 예: `mfg:Product1`, `mfg:Product2` 등입니다. 접두사 `mfg:`는 전체 데이터베이스를 뜻하므로, 이름은 전 세계에서 고유합니다.
2. **열 하나는 속성 하나가 됩니다.** 표 이름과 열 이름을 연결합니다. 예: `mfg:Product_ModelNo`, `mfg:Product_Division`입니다. 이렇게 하면 둘 다 "name"이라는 열을 가진 두 표를 혼동하지 않습니다.
3. **셀 하나는 트리플 하나가 됩니다.** 트리플의 목적어는 일반 값(문자열 또는 숫자)입니다. 일반 값은 리터럴(literal)이라고 합니다. 리터럴은 자체 URI를 가진 사물이 아닙니다.
4. **유형을 추가합니다.** 각 행에 `mfg:Product1 rdf:type mfg:Product`와 같은 트리플 하나를 추가합니다.

계산: n개 행 × c개 열 = n × c개 트리플입니다. 여기서는 9 × 7 = 63개 트리플이며, 여기에 유형 트리플 9개가 더해집니다.

주어	서술어	목적어 (리터럴)
mfg:Product1	mfg:Product_ModelNo	ZX-3
mfg:Product1	mfg:Product_Division	Manufacturing support
mfg:Product1	mfg:Product_Available	23
mfg:Product1	rdf:type	mfg:Product

7. 사실에 관한 사실과 이름 붙은 그래프(named graph)

한 줄 요약: "누가 그렇게 말했는가"까지 적고 싶을 때 쓰는 도구입니다.

때로는 트리플 하나로 충분하지 않습니다. "Shakespeare가 1601년에 Hamlet을 썼다" 또는 "Wikipedia는 Shakespeare가 Hamlet을 썼다고 말한다"라고 표현하고 싶을 수 있습니다. 진술에 대해 말하는 것을 진술에 대한 진술(reification)이라고 합니다. 두 가지 경우가 있습니다.

경우 A: 사건에 세부 정보를 추가합니다. 사건 자체를 하나의 사물로 만든 다음, 그 사물에 세부 정보를 연결합니다. 예를 들어 저자가 Shakespeare이고, 제목이 "Hamlet"이며, 출판 날짜가 1601인 `n1`이라는 사물입니다. 이것은 표의 행과 같은 생각입니다.

경우 B: 주장에 대해 주장합니다. RDF에는 이를 위한 세 가지 특별한 단어가 있습니다. `rdf:subject`, `rdf:predicate`, `rdf:object`입니다. 이들은 트리플을 하나의 사물로 설명합니다. 그러면 "Wikipedia가 그 사물에 대해 말한다"라고 표현할 수 있습니다. 중요합니다. 진술을 설명한다고 해서 그 진술이 참이라는 뜻은 아닙니다. Wikipedia를 신뢰하지 않는 프로그램은 "Shakespeare가 Hamlet을 썼다"를 사실로 처리하지 않습니다.

또 다른 도구는 이름 붙은 그래프(named graph)입니다. 그래프는 트리플의 집합일 뿐입니다. 이름 붙은 그래프는 이름(URI)이 붙은 트리플 집합입니다. 세 가지 상황에서 사용합니다.

- 파일 하나, 그래프 하나: 각 출처의 데이터를 각각의 묶음으로 보관합니다. 묶음 이름은 데이터가 어디에서 왔는지도 알려 줄 수 있습니다.
- 진술에 대한 진술(reification): 트리플을 이름 붙은 그래프에 넣고, 그 그래프에 대해 말합니다.
- 맥락: 어떤 상황에서 진술이 성립하는지 보여 줍니다. 예: "그 영화에서"입니다.

트리플에 그래프 이름을 더하면 네 부분이 되며, 이를 쿼드(quad)라고 합니다. 즉, 주어, 서술어, 목적어, 그래프입니다.

8. RDF를 쓰는 방법: 파일 형식

한 줄 요약: 같은 내용을 여러 글씨체로 쓸 수 있습니다. 이 수업에서는 Turtle을 씁니다.

같은 문장을 인쇄체나 필기체로 쓸 수 있듯이, 같은 트리플도 여러 형식으로 쓸 수 있습니다. 형식 사이에서 변환해도 정보는 사라지지 않습니다.

형식	한 줄 설명
N-Triples	가장 단순합니다. 한 줄에 트리플 하나를 쓰고, 꺾쇠괄호 안에 전체 URI를 쓰며, 마침표로 끝냅니다. 프로그램이 처리하기 쉽습니다.
Turtle	접두사를 사용하므로 짧고 읽기 쉽습니다. 이 수업의 주요 형식입니다.
RDF/XML	W3C가 초기에 권장한 XML 버전입니다.
JSON-LD	RDF와 맞는 JSON입니다. 이미 JSON을 아는 웹 개발자에게 편리합니다.
N-Quads	그래프 이름이 추가된 N-Triples입니다.
TriG	중괄호 안에 이름 붙은 그래프를 넣은 Turtle입니다.

Turtle 요약표:

기호	뜻
`@prefix`	접두사와 그것이 나타내는 네임스페이스를 선언합니다
`.`	트리플의 끝입니다
`;`	같은 주어이며, 다음 서술어와 목적어가 이어집니다
`,`	같은 주어와 서술어이며, 다음 목적어가 이어집니다
`a`	`rdf:type`의 줄임말입니다

예: `lit:Shakespeare a lit:Playwright ; b:hasChild b:Susanna , b:Judith , b:Hamnet .` 이것은 네 개의 트리플입니다. 유형 트리플 하나와 `hasChild` 트리플 세 개입니다.

9. 빈 노드(blank node)와 순서 있는 목록

한 줄 요약: 빈 노드는 "누군가 있는데 이름은 모른다"를 뜻합니다.

빈 노드는 "여기에 무언가가 있지만, ID를 모르거나 알 필요가 없다"를 뜻합니다. 예: "Shakespeare의 시는 England에 살았던 한 여성에게서 영감을 받았지만, 그 여성이 누구인지는 모른다"입니다. Turtle에서 빈 노드는 대괄호로 씁니다. `[ a bio:Woman ; bio:livedIn geo:England ]`처럼 씁니다. 빈 노드에는 URI가 없으므로 다른 사람이 이를 참조할 수 없습니다. 따라서 일반적으로 권장하지 않습니다. 단, OWL은 특별한 방식으로 이를 사용합니다.

트리플에는 순서가 없습니다. 순서가 필요하면, 예를 들어 자녀의 순서가 필요하면, Turtle에서는 둥근괄호를 씁니다. `b:hasChild ( b:Susanna b:Judith b:Hamnet )`처럼 씁니다. 내부적으로 이 목록은 `rdf:first`(현재 항목), `rdf:rest`(목록의 나머지), `rdf:nil`(끝)로 연결된 빈 노드의 사슬입니다.

10. 요약과 복습 질문

RDF는 웹 전체에 퍼진 데이터를 설명하는 간단한 표준입니다. 핵심 생각은 이렇습니다. 분산된 데이터에 대해 말하려면 각 사물, 각 관계, 각 값의 연결에 전역 이름이 필요합니다. 웹은 이미 전역 이름(URI)을 제공하므로, RDF는 이를 다시 사용합니다. 그 밖의 모든 것(파일 형식, 표 변환)은 트리플 주변의 처리 과정입니다.

핵심 용어: 트리플, 그래프, 병합, URI, 네임스페이스, CURIE, rdf:type, rdf:Property, 진술에 대한 진술(reification), 이름 붙은 그래프(named graph), 직렬화(serialization), 빈 노드(blank node).

복습 질문:

1. 데이터가 행, 열, 셀로 나뉘어 있을 때 전역적으로 합의해야 하는 것은 무엇입니까?
2. 두 그래프의 두 노드는 언제 같은 노드로 처리됩니까?
3. 표에 n개 행과 c개 열이 있습니다. 유형 트리플을 제외하면 트리플은 몇 개입니까?
4. "Wikipedia는 Shakespeare가 Hamlet을 썼다고 말한다"를 표현하는 두 가지 방법을 말해 보세요.
5. 진술을 설명한다고 해서 그 진술이 참이 되지 않는 이유는 무엇입니까?
6. Turtle에서 `;`, `,`, `a`, `[ ]`, `( )`는 무엇을 뜻합니까?
7. 빈 노드는 언제 사용하며, 일반적으로 권장하지 않는 이유는 무엇입니까?

11. 지식 그래프란 무엇인가요?

한 줄 요약: 지식 그래프는 "점(사물)과 선(관계)"으로 지식을 저장한 아주 큰 RDF 모음입니다.

지식 그래프는 사물(노드)과 그 사이의 관계(엣지)를 그래프 형태로 저장합니다. RDF는 지식 그래프를 만드는 주요 방법 중 하나입니다. 지식 그래프는 수집되고, 연결되고, 검색할 수 있게 만든 매우 큰 RDF 트리플의 모음이라고 생각할 수 있습니다.

지식 그래프의 구성 요소:

- 엔터티(노드): 사람, 장소, 조직, 아이디어입니다.
- 관계(엣지): 엔터티가 연결되는 방식입니다(RDF의 서술어).
- 속성 값: 이름, 나이, 날짜와 같은 리터럴(literal)입니다.
- 어휘(vocabulary)(스키마 또는 온톨로지): FOAF 또는 schema.org처럼 클래스와 속성에 대해 합의된 의미입니다. 이를 통해 서로 다른 사람의 데이터를 함께 맞출 수 있습니다.
- 저장소와 질의 언어: SPARQL을 사용하는 트리플 저장소(triple store), 또는 Cypher를 사용하는 Neo4j 같은 속성 그래프 데이터베이스입니다.

그래프의 일반적인 두 종류는 다음과 같습니다.

	RDF 그래프	속성 그래프(LPG)
기본 단위	트리플(주어 - 서술어 - 목적어)	속성을 가질 수 있는 레이블된 노드와 관계
식별성	전역 URI	하나의 데이터베이스 안의 ID
관계의 세부 정보	진술에 대한 진술(reification) 또는 이름 붙은 그래프가 필요합니다	속성을 직접 붙일 수 있습니다
질의 언어	SPARQL	Cypher, GQL
강점	데이터 공유와 결합, 표준 어휘, 추론	쉬운 경로 탐색과 앱 개발

어느 쪽도 모든 경우에 더 좋은 것은 아닙니다. RDF는 주로 공유와 통합을 위해 선택되며, 속성 그래프는 하나의 애플리케이션 안에서 검색할 때 주로 선택됩니다.

공개 지식 그래프: DBpedia(Wikipedia 데이터를 RDF로 만든 것)와 Wikidata(Wikimedia Foundation이 운영하는 협업 그래프이며, SPARQL 질의 서비스를 제공합니다)입니다.

12. 최근 동향: LLM과 지식 그래프의 만남

한 줄 요약: GraphRAG는 LLM이 답하기 전에 지식 그래프에서 "연결된 사실"을 찾아 주는 방법입니다.

대규모 언어 모델(LLM)은 자연스러운 답변을 쓸 수 있지만, 때로는 틀린 내용을 말하기도 합니다(환각, hallucination). 또한 여러분의 비공개 데이터는 알지 못합니다. RAG(검색 증강 생성, retrieval-augmented generation)는 먼저 문서를 찾아 모델에 제공하여 이를 돕습니다. GraphRAG는 한 단계 더 나아가 지식 그래프에서 연결된 사실을 찾습니다.

GraphRAG의 세 단계(2024년 설문 논문 기준):

1. 공개 또는 비공개 데이터로 그래프를 만들고 인덱싱합니다.
2. 질문과 관련된 그래프 조각(노드, 트리플, 경로, 작은 부분 그래프)을 검색합니다.
3. 이 조각들을 텍스트로 바꾸고, 모델이 답변을 쓰도록 제공합니다.

일반 RAG와 비교한 장점: 엔터티 사이의 관계를 유지하고, 요약을 통해 반복을 줄이며, 전체 데이터 세트에 대한 넓은 질문에 답할 수 있습니다. 한계: 그래프가 커질수록 가능한 부분 그래프의 수가 폭발적으로 늘어나므로 효율적인 검색이 필요합니다. 또한 텍스트 질문이 그래프 형태와 얼마나 잘 맞는지 측정하기 어렵습니다.

2026년 한 논문은 펀드 데이터에 관한 200개 질문에서 세 가지 방법을 비교했습니다. 200점 만점의 점수는 임베딩을 사용한 에이전트형 RAG가 116점, RDF 기반 GraphRAG가 172.5점, 속성 그래프 기반 GraphRAG가 185.5점이었습니다. 이는 한 분야에서 진행한 하나의 연구이므로, 너무 일반화하면 안 됩니다. 이 연구의 조언은 다음과 같습니다. 복잡한 여러 단계의 검색에는 속성 그래프를 사용하고, 질의 패

턴을 예측하기 어려울 때는 RDF를 사용합니다. 엔터티와 속성은 별도의 노드로 모델링합니다. 구조화된 데이터를 일반 문장으로 바꾸지 않아야 합니다. 이렇게 하면 정보가 사라지고 오류가 생길 수 있습니다. 질문을 Cypher 질의로 바꾸는 일도 여전히 오류가 발생하기 쉽습니다.

다른 동향(일반적인 배경 정보이며, 두 논문에서 나온 내용은 아닙니다):

- LLM을 사용하여 문서에서 엔터티와 관계를 추출하고, 이를 트리플로 저장합니다. 오류가 생길 수 있으므로 확인 단계가 필요합니다.
- SHACL은 그래프가 규칙(필수 속성, 허용된 값)을 따르는지 확인하는 W3C 표준입니다.
- GQL은 RDF의 SPARQL 다음으로 나온 속성 그래프용 ISO 표준 질의 언어입니다.

출처: Graph Retrieval-Augmented Generation: A Survey (arXiv 2408.08921) · Graphs RAG at Scale: LPG and RDF (arXiv 2603.22340)

13. Python 실습 1: FOAF로 RDF 작성하기

목표: Python으로 작은 "누가 누구를 아는가" 그래프를 만들고 Turtle로 출력합니다. FOAF(Friend of a Friend)는 사람을 위한 미리 만들어진 어휘입니다. Python 라이브러리 rdflib를 사용합니다(설치: `pip install rdflib`).

```
from rdflib import Graph, Namespace, Literal, URIRef, BNode, RDF, XSD
from rdflib.namespace import FOAF

EX = Namespace("http://example.org/people#")
g = Graph()
g.bind("foaf", FOAF); g.bind("ex", EX)

# 사람 3명: 타입, 이름, 나이
for pid, name, age in [(EX.alice, "Alice Kim", 24),
                       (EX.bob, "Bob Lee", 31),
                       (EX.carol, "Carol Park", 28)]:
    g.add((pid, RDF.type, FOAF.Person))
    g.add((pid, FOAF.name, Literal(name)))
    g.add((pid, FOAF.age, Literal(age, datatype=XSD.integer)))

g.add((EX.alice, FOAF.mbox, URIRef("mailto:alice@example.org")))

# 관계: Alice -> Bob -> Carol
g.add((EX.alice, FOAF.knows, EX.bob))
g.add((EX.bob, FOAF.knows, EX.carol))

# 블랭크 노드: 식별자를 모르는 지인
anon = BNode()
g.add((EX.carol, FOAF.knows, anon))
```

```
g.add((anon, RDF.type, FOAF.Person))
g.add((anon, FOAF.name, Literal("Dave (unknown ID)")))

print(g.serialize(format="turtle"))
```

쉬운 말로 설명하면: 사람 세 명을 만들고, 각 사람에게 이름과 나이를 주고, 서로 연결합니다(Alice는 Bob을 알고, Bob은 Carol을 압니다). 또한 ID를 모르는 사람 한 명(빈 노드)도 추가합니다. 그다음 모든 내용을 Turtle로 출력합니다.

출력(일부):

```
ex:alice a foaf:Person ;
    foaf:age 24 ;
    foaf:knows ex:bob ;
    foaf:mbox <mailto:alice@example.org> ;
    foaf:name "Alice Kim" .

ex:carol a foaf:Person ;
    foaf:age 28 ;
    foaf:knows [ a foaf:Person ;
            foaf:name "Dave (unknown ID)" ] ;
    foaf:name "Carol Park" .
```

한 줄씩 살펴보기

- `from rdflib import ...` : 그래프, 네임스페이스, 리터럴(일반 값), URI, 빈 노드, 그리고 `rdf:` / `xsd:` 단어 도구를 가져옵니다.
- `from rdflib.namespace import FOAF` : 미리 만들어진 FOAF 어휘를 가져옵니다. 따라서 `FOAF.name` 은 공식 FOAF "이름"을 뜻합니다.
- `EX = Namespace(...)` : 우리가 만든 네임스페이스입니다. `EX.alice` 는 전체 URI `http://example.org/people#alice` 가 됩니다.
- `g = Graph()` : 비어 있는 트리플 묶음입니다. 같은 트리플을 두 번 추가해도 하나만 남습니다.
- `g.bind(...)` : 출력에서 긴 URI 대신 `foaf:` 가 보이도록 별명(접두사)을 등록합니다.
- `for pid, name, age in [...]` : 사람 세 명을 한 명씩 살펴봅니다.
- `g.add((pid, RDF.type, FOAF.Person))` : "이것은 사람(Person)입니다."
- `g.add((pid, FOAF.name, Literal(name)))` : 이름을 일반 텍스트 값으로 붙입니다.
- `g.add((pid, FOAF.age, Literal(age, datatype=XSD.integer)))` : 나이를 정수로 붙입니다. 숫자라고 표시하면 나중에 나이를 비교할 수 있습니다(예: "25세 이상").
- `g.add((EX.alice, FOAF.mbox, URIRef("mailto:...")))` : FOAF는 이메일 주소를 식별자로 다루므로, 이메일은 `mailto:` URI로 저장합니다. Alice에게만 이메일을 추가합니다. 나중에 `OPTIONAL` 을 보여 줄 때 사용합니다.

- `g.add((EX.alice, FOAF.knows, EX.bob))`와 다음 줄: Alice는 Bob을 알고, Bob은 Carol을 압니다. 화살표에는 방향이 있습니다. Bob이 Alice를 안다는 뜻은 아닙니다.
- `anon = BNode()`: ID가 없는 "누군가"를 만듭니다.
- `anon`을 사용하는 세 줄: Carol은 이 누군가를 압니다. 이 누군가는 "Dave (unknown ID)"라는 이름의 Person입니다.
- `print(g.serialize(format="turtle"))`: 전체 그래프를 Turtle로 출력합니다. 같은 주어에 대한 트리플은 `;`로 이어지고, 빈 노드는 `[ ... ]`로 나타납니다.

직접 해 보기: 네 번째 사람을 추가하고, Carol이 그 사람을 알도록 만든 뒤 다시 출력해 보세요.

14. Python 실습 2: 지식 그래프 만들기

목표: 작은 표(CSV)를 그래프로 바꾸고, 그다음 분리된 그래프 두 개를 합칩니다.

파트 A: 표에서 트리플로

```
import csv, io
from rdflib import Graph, Namespace, Literal, RDF
from rdflib.namespace import FOAF

data = """id,name,dept,advisor
s1,Minji,BigData,p1
s2,Joon,BigData,p1
s3,Sora,Business,p2
"""
KG = Namespace("http://example.org/khu#")
k = Graph(); k.bind("kg", KG); k.bind("foaf", FOAF)

# 교수 정보
k.add((KG.p1, RDF.type, KG.Professor)); k.add((KG.p1, FOAF.name,
Literal("Prof. Han")))
k.add((KG.p2, RDF.type, KG.Professor)); k.add((KG.p2, FOAF.name,
Literal("Prof. Seo")))

# 학생 표 -> 트리플
for r in csv.DictReader(io.StringIO(data)):
    s = KG[r["id"]]                                  # 행 -> 자원 URI
    k.add((s, RDF.type, KG.Student))                  # 테이블 이름 -> 타입
    k.add((s, FOAF.name, Literal(r["name"])))         # 열 -> 속성(리터럴)
    k.add((s, KG.department, KG[r["dept"]]))          # 열 -> 관계(자원)
    k.add((s, KG.advisedBy, KG[r["advisor"]]))

print("KG 트리플 수:", len(k))                 # 실행 결과: 16
```

하는 일: 각 학생 행은 하나의 대상이 되고, 각 열은 하나의 속성이 됩니다. 학과와 지도교수는 다른 대상으로 연결됩니다.

한 줄씩 살펴보기

- `import csv, io`: CSV 텍스트를 읽는 도구입니다. 실제 파일이라면 `io.StringIO(data)` 대신 `open("students.csv")`를 사용합니다.
- `data = """id,name,dept,advisor ..."""`: 텍스트로 만든 작은 표입니다. 첫 줄에는 열 이름이 있고, 다음 세 줄에는 학생 세 명이 있습니다.
- `KG = Namespace(...)`: 우리가 만든 네임스페이스입니다. 우리가 만드는 모든 클래스, 대상, 속성은 이 아래에 있습니다.
- `k = Graph(); k.bind(...)`: 비어 있는 그래프와 보기 좋은 출력을 위한 접두사입니다.
- `KG.p1`(그리고 `p2`)을 위한 두 `k.add` 줄: 학생들이 이 교수들을 가리킬 수 있도록, 먼저 유형과 이름을 가진 교수들을 추가합니다.
- `for r in csv.DictReader(...)`: 표를 한 번에 한 행씩 읽습니다. `r["name"]`은 "name" 열의 값을 줍니다.
- `s = KG[r["id"]]`: 행 ID가 학생의 URI가 됩니다(6장의 규칙 1).
- `k.add((s, RDF.type, KG.Student))`: 표의 의미("이것은 학생입니다")가 유형 트리플이 됩니다.
- `k.add((s, FOAF.name, Literal(r["name"])))`: "name" 열은 속성이 되고, 셀은 일반 값이 됩니다. 새 이름을 만들지 않고 FOAF에 있는 기존 "name"을 다시 사용합니다.
- `department`와 `advisedBy` 줄: 이 셀들은 일반 텍스트가 아니라 다른 대상(URI)으로 가는 연결입니다. 따라서 그래프는 학생을 교수와 연결합니다.
- `print(len(k))`: 트리플 수를 셉니다. 각 학생은 4개의 트리플(유형, 이름, 학과, 지도교수)을 만들고, 각 교수는 2개(유형, 이름)를 만듭니다. 따라서 3×4 + 2×2 = 16입니다.

파트 B: 그래프 두 개 합치기

```
from rdflib import Namespace, Graph, RDF
LIT = Namespace("http://example.org/lit#")
GEO = Namespace("http://example.org/geo#")
BIO = Namespace("http://example.org/bio#")

g_lit = Graph(); g_geo = Graph()
g_lit.add((LIT.Shakespeare, RDF.type, LIT.Playwright))
g_lit.add((LIT.Shakespeare, LIT.wrote, LIT.Macbeth))
g_lit.add((LIT.Macbeth, LIT.setIn, GEO.Scotland))
g_lit.add((LIT.Shakespeare, BIO.livedIn, GEO.Stratford))
g_geo.add((GEO.Stratford, GEO.isIn, GEO.England))
g_geo.add((GEO.England, GEO.partOf, GEO.UK))
g_geo.add((GEO.Scotland, GEO.partOf, GEO.UK))

kg = g_lit + g_geo          # 병합
print(len(g_lit), len(g_geo), len(kg))   # 4 3 7
```

하는 일: 한 그래프는 문학 정보를 알고, 다른 그래프는 지리 정보를 압니다. 두 그래프는 `GEO.Scotland`와 `GEO.Stratford`라는 이름을 함께 사용합니다. 그래프를 합치면, 이 공유 이름들이 두 세계를 연결하는 하나의 그래프가 됩니다.

한 줄씩 살펴보기

- `from rdflib import ...`: 네임스페이스, 그래프, `rdf:type`을 위한 도구입니다.
- `LIT`, `GEO`, `BIO`: 문학, 지리, 전기를 위한 세 개의 네임스페이스입니다.
- `g_lit = Graph(); g_geo = Graph()`: 서로 다른 두 출처를 나타내는 그래프 두 개입니다.
- 네 개의 `g_lit.add` 줄: Shakespeare는 극작가입니다. 그는 Macbeth를 썼습니다. Macbeth의 배경은 Scotland입니다. 그는 Stratford에서 살았습니다. Scotland와 Stratford에는 `GEO` 네임스페이스를 사용한다는 점에 주의하세요. 이것이 연결 지점입니다.
- 세 개의 `g_geo.add` 줄: Stratford는 England에 있고, England와 Scotland는 UK의 일부입니다. 위와 같은 `GEO` 이름을 사용합니다.
- `kg = g_lit + g_geo`: 두 묶음을 새 그래프로 합칩니다. 같은 URI를 가진 노드는 자동으로 하나의 노드가 됩니다. 원래 그래프는 바뀌지 않습니다.
- `print(len(g_lit), len(g_geo), len(kg))`: 4, 3, 7을 출력합니다. 반복된 트리플이 없으므로 4 + 3 = 7입니다.

15. Python 실습 3: SPARQL로 질문하기

목표: 앞의 실습에서 만든 그래프를 검색합니다. SPARQL은 RDF의 표준 질의 언어(query language)입니다. 질의는 "빈칸이 있는 패턴"을 적는 것입니다. `?`로 시작하는 이름(예: `?name`)이 빈칸입니다. 그래프는 패턴에 맞게 빈칸을 채울 수 있는 모든 방법을 돌려줍니다.

먼저 도우미(아래 모든 질의에서 사용합니다). 13장의 그래프 `g`와 14장의 그래프 `k`가 이미 만들어져 있어야 합니다.

```
P = "PREFIX foaf: <http://xmlns.com/foaf/0.1/>\nPREFIX ex:
<http://example.org/people#>\n"

def run(title, graph, q):
    print(f"## {title}")
    for row in graph.query(q):
        print(" | ".join(str(x) if x is not None else "-" for x in row))
```

한 줄씩 살펴보기

- `P = "PREFIX foaf: <...>\nPREFIX ex: <...>\n"` : Turtle의 `@prefix`와 같은 접두사 선언입니다. 각 질의 앞에 넣습니다. `\n`은 새 줄을 뜻합니다.
- `def run(title, graph, q):` : 제목, 그래프, 질의를 받는 도우미입니다.
- `print(f"## {title}")` : 먼저 제목을 출력합니다.
- `for row in graph.query(q):` : 질의를 실행합니다. 각 결과 행에는 `SELECT`의 순서대로 빈칸의 값이 들어 있습니다.
- `print(" | ".join(...))` : `|`로 구분하여 값을 출력합니다. 값이 없으면(`OPTIONAL`에서는 그럴 수 있습니다) `-`를 출력합니다.

일곱 가지 질문

```
# (1) SELECT + ORDER BY: 이름과 나이
run("나이 순", g, P + """
SELECT ?name ?age WHERE { ?p a foaf:Person ; foaf:name ?name ; foaf:age ?age
}
ORDER BY ?age""")

# (2) FILTER: 25세 이상
run("25세 이상", g, P + """
SELECT ?name WHERE { ?p foaf:name ?name ; foaf:age ?a . FILTER(?a >= 25)
}""")

# (3) OPTIONAL: 이메일이 없어도 행을 유지
run("이메일(선택)", g, P + """
SELECT ?name ?mbox WHERE {
  ?p a foaf:Person ; foaf:name ?name .
  OPTIONAL { ?p foaf:mbox ?mbox } } ORDER BY ?name""")

# (4) 속성 경로: knows 를 1회 이상 따라감
run("Alice의 직간접 지인", g, P + """
SELECT ?name WHERE { ex:alice foaf:knows+ ?p . ?p foaf:name ?name } ORDER BY
```

```
?name""")

# (5) 집계: 지도교수별 학생 수 (14장의 그래프 k)
run("지도교수별 학생 수", k, """
PREFIX kg: <http://example.org/khu#>
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
SELECT ?prof (COUNT(?s) AS ?n) WHERE { ?s kg:advisedBy ?a . ?a foaf:name ?
prof }
GROUP BY ?prof ORDER BY DESC(?n)""")

# (6) ASK: 참/거짓
print(g.query(P + "ASK { ex:alice foaf:knows ex:bob }").askAnswer)   # True

# (7) CONSTRUCT: 질의 결과로 새 그래프 생성
c = g.query(P + """CONSTRUCT { ?b foaf:knownBy ?a }
WHERE { ?a foaf:knows ?b . ?b a foaf:Person . ?a foaf:name ?x }""")
print(len(c.graph))   # 3
```

실행 결과 요약:

질문	결과
(1) 나이 순	Alice Kim 24, Carol Park 28, Bob Lee 31
(2) 25세 이상	Bob Lee, Carol Park
(3) 이메일(선택)	4행. Alice만 이메일이 있고 나머지는 `-`
(4) Alice의 직간접 지인	Bob Lee, Carol Park, Dave (unknown ID)
(5) 지도교수별 학생 수	Prof. Han 2, Prof. Seo 1
(6) ASK	True
(7) CONSTRUCT	트리플 3개

한 줄씩 살펴보기

(1) SELECT + ORDER BY: 이름과 나이를 어린 사람부터 나열합니다.

- `SELECT ?name ?age` : 돌려받고 싶은 빈칸입니다.
- `WHERE { ?p a foaf:Person ; foaf:name ?name ; foaf:age ?age }` : 이름과 나이가 있는 사람 `?p` 를 찾습니다. `;` 는 "같은 주어가 계속됩니다"라는 뜻입니다. Dave는 나이가 없으므로 맞지 않아 제외됩니다.
- `ORDER BY ?age` : 나이순으로 정렬합니다. 반대 순서는 `DESC(?age)` 를 씁니다.

(2) FILTER: 25세 이상인 사람만 찾습니다.

- 패턴은 이름과 나이를 찾고, `FILTER(?a >= 25)`는 조건을 통과한 행만 남깁니다. Bob(31)과 Carol(28)은 남고, Alice(24)는 제외됩니다. 나이가 숫자로 저장되었기 때문에 가능합니다.

(3) OPTIONAL: 이메일이 없어도 모든 사람을 보여 줍니다.

- 첫 번째 패턴은 이름이 있는 모든 사람을 찾습니다.
- `OPTIONAL { ?p foaf:mbox ?mbox }`: 이메일이 있으면 채우고, 없으면 행을 그대로 둡니다. `OPTIONAL`이 없다면 이메일이 있는 Alice만 나옵니다.

(4) 속성 경로(property path): 거리에 상관없이 친구의 친구를 찾습니다.

- `ex:alice foaf:knows+ ?p`: `+`는 "한 번 이상"이라는 뜻입니다. Alice에서 시작하여 `knows` 연결을 갈 수 있는 데까지 따라갑니다. Bob, Carol, Dave가 나옵니다. (`*`는 0번 이상, `?`는 0번 또는 1번입니다.)
- `?p foaf:name ?name`: 도달한 각 사람의 이름을 가져옵니다.

(5) 개수 세기: 각 교수는 몇 명의 학생을 지도할까요? (14장의 그래프 `k` 사용)

- 추가 `PREFIX` 줄: 이 질의는 학생 그래프를 쓰므로 그 접두사가 필요합니다.
- `SELECT ?prof (COUNT(?s) AS ?n)`: 교수의 이름과 `?n`이라는 개수를 돌려줍니다.
- `WHERE { ?s kg:advisedBy ?a . ?a foaf:name ?prof }`: 학생에서 지도교수로 가는 연결을 따라간 뒤 지도교수의 이름을 읽습니다. 14장에서 지도교수를 "연결"로 저장했기 때문에 가능합니다.
- `GROUP BY ?prof ORDER BY DESC(?n)`: 교수별로 세고, 큰 수부터 보여 줍니다(Prof. Han 2, Prof. Seo 1).

(6) ASK: 예/아니요 질문입니다.

- `ASK { ex:alice foaf:knows ex:bob }`: 이 트리플이 그래프에 있나요?
- `.askAnswer`: True/False 결과를 읽습니다. 여기서는 `True`입니다.

(7) CONSTRUCT: 답에서 새 그래프를 만듭니다.

- `CONSTRUCT { ?b foaf:knownBy ?a }`: 새 트리플의 틀입니다. "knows"의 반대 방향입니다.
- `WHERE { ?a foaf:knows ?b . ?b a foaf:Person . ?a foaf:name ?x }`: `?a`가 `?b`를 알고, `?b`가 Person이며, `?a`가 이름을 가진 쌍을 가져옵니다. (`?x`는 검사에만 씁니다.)
- `c.graph`와 `len(...)`: 새 그래프에는 트리플이 3개 있습니다(Alice→Bob, Bob→Carol, Carol→Dave가 각각 반대 방향으로 만들어집니다).

직접 해 보기: 누군가를 아는 사람만 돌려주는 질의를 작성해 보세요.

16. Python 실습 4: 작은 GraphRAG

목표: 약 50줄로 GraphRAG의 아이디어를 살펴봅니다. 문장을 트리플로 바꾸고, 그래프를 만들고, 질문 근처의 사실만 찾고, LLM에 보낼 텍스트를 만듭니다. API 키는 필요하지 않습니다. 설치: `pip install networkx rdflib`.

작동 방식:

1. 문장을 넣으면 트리플이 나옵니다. (여기서는 간단한 패턴이 이 일을 합니다. 실제 시스템에서는 LLM이 이 단계를 합니다.)
2. 트리플을 그래프에 넣습니다.
3. 질문에서 언급한 대상을 찾고, 3단계 안에 있는 모든 것을 모읍니다.
4. 찾은 사실이 LLM에게 줄 프롬프트(prompt)가 됩니다.
5. 또한 그래프를 커뮤니티(community), 즉 촘촘하게 연결된 대상들의 그룹으로 나눕니다. 이것은 전체 데이터에 관한 넓은 질문에 답하는 데 도움이 됩니다.

```
import re
import networkx as nx
from networkx.algorithms.community import greedy_modularity_communities
from rdflib import Graph, Namespace, Literal, RDFS

EX = Namespace("http://example.org/kg#")

# 1) 원천 문서(비정형 텍스트)
docs = [
    "Shakespeare wrote Hamlet.",
    "Shakespeare wrote Macbeth.",
    "Shakespeare lived in Stratford.",
    "Stratford is in England.",
    "Macbeth is set in Scotland.",
    "England is part of the UK.",
    "Scotland is part of the UK.",
    "Marlowe wrote Edward II.",
    "Marlowe lived in Canterbury.",
    "Canterbury is in England.",
]

# 2) 트리플 추출 (실전에서는 이 함수를 LLM 호출로 교체)
REL = {"wrote": "wrote", "lived in": "livedIn", "is in": "isIn",
       "is set in": "setIn", "is part of": "partOf"}
PATTERN = re.compile(r"^(.+?) (wrote|lived in|is in|is set in|is part of)
(.+?)\.$")

def extract_triples(text):
    m = PATTERN.match(text)
```

```
    return (m.group(1), REL[m.group(2)], m.group(3)) if m else None

triples = [t for t in map(extract_triples, docs) if t]

# 3) 지식 그래프 구축 (rdflib: 저장·SPARQL, networkx: 탐색)
def uri(label): return EX[label.replace(" ", "_")]
kg = Graph(); kg.bind("ex", EX)
nxg = nx.Graph()
for s, p, o in triples:
    kg.add((uri(s), EX[p], uri(o)))
    kg.add((uri(s), RDFS.label, Literal(s)))
    kg.add((uri(o), RDFS.label, Literal(o)))
    nxg.add_edge(s, o, rel=p)
print("트리플 수(관계만):", len(triples), "/ 노드 수:", nxg.number_of_nodes())

# 4) 질문 -> 엔티티 연결 -> k-hop 부분 그래프 검색
def retrieve(question, hops=3):
    seeds = [n for n in nxg.nodes if n.lower() in question.lower()]
    nodes = set(seeds)
    for s in seeds:
        nodes |= set(nx.single_source_shortest_path_length(nxg, s,
cutoff=hops))
    sub = nxg.subgraph(nodes)
    facts = []
    for s, p, o in triples:
        if s in sub and o in sub:
            facts.append(f"{s} {p} {o}")
    return seeds, facts

question = "Where did Shakespeare live, and which country is that part of?"
seeds, facts = retrieve(question, hops=3)
print("시드 엔티티:", seeds)
print("검색된 사실 수:", len(facts))

# 5) 컨텍스트를 프롬프트로 구성 (LLM 호출 직전 단계)
prompt = (
    "Answer using ONLY the facts below.\n"
    "Facts:\n" + "\n".join(f"- {f}" for f in facts) +
    f"\n\nQuestion: {question}\nAnswer:"
)
print(prompt)

# 6) 전역 질문용: 커뮤니티 탐지 (Microsoft GraphRAG의 아이디어)
for i, c in enumerate(greedy_modularity_communities(nxg), 1):
    print(i, sorted(c))
```

예시 출력:

```
트리플 수(관계만): 10 / 노드 수: 10
시드 엔티티: ['Shakespeare']
검색된 사실 수: 8

Facts:
- Shakespeare wrote Hamlet
- Shakespeare wrote Macbeth
- Shakespeare livedIn Stratford
- Stratford isIn England
- Macbeth setIn Scotland
- England partOf the UK
- Scotland partOf the UK
- Canterbury isIn England

커뮤니티
1 ['Canterbury', 'Edward II', 'England', 'Marlowe']
2 ['Hamlet', 'Shakespeare', 'Stratford']
3 ['Macbeth', 'Scotland', 'the UK']
```

트리플 10개, 노드 10개입니다. 질문에서 찾은 시작점은 Shakespeare입니다. 10개의 사실 중 8개를 가져옵니다.

출력이 알려 주는 점

- 답으로 가는 경로는 Shakespeare - Stratford - England - the UK입니다.
- "Canterbury isIn England"도 가져오지만, 필요하지는 않습니다. England에서 한 단계 거리이기 때문입니다. 더 넓게 검색하면 유용한 사실도 더 찾지만 잡음도 더 많아집니다. 이것이 GraphRAG의 기본적인 상충 관계(trade-off)입니다.
- 문장에서 그 철자를 사용했기 때문에 "the UK"는 정확히 그 철자의 노드 하나로 저장됩니다. 실제 시스템에는 같은 대상을 다른 철자로 쓴 것을 합치는 엔터티 정규화(entity normalization)가 필요합니다.

한 줄씩 살펴보기

- `import re` : 정규 표현식(regular expressions)입니다. 문장에서 부분을 골라내는 데 씁니다.
- `import networkx as nx` : 그래프와 노드 사이의 거리를 재는 라이브러리입니다.
- `from networkx.algorithms.community import greedy_modularity_communities` : 촘촘하게 연결된 노드를 그룹으로 나누는 함수입니다.
- `from rdflib import ...` : 앞의 실습에서 쓴 RDF 도구입니다.
- `EX = Namespace(...)` : 우리의 네임스페이스(namespace)입니다.
- `docs = [...]` : 원본 텍스트를 나타내는 문장 10개입니다. 문장마다 사실 하나가 있습니다.
- `REL = {...}` : 구절("lived in")을 속성 이름(`livedIn`)으로 바꾸는 사전(dictionary)입니다.

- `PATTERN = re.compile(...)` : 문장의 틀입니다. 주어, 알려진 구절 중 하나, 마침표로 끝나는 목적어입니다. 괄호 세 쌍이 이 세 부분을 잡아냅니다.
- `def extract_triples(text):` : 문장을 받아 트리플을 돌려줍니다. 실제 환경에서는 이 함수만 LLM 호출로 바꾸면 됩니다.
- `m = PATTERN.match(text)` : 문장이 이 틀에 맞나요?
- `return (...) if m else None` : 맞으면 (주어, 속성 이름, 목적어)를 돌려주고, 맞지 않으면 아무것도 돌려주지 않습니다.
- `triples = [...]` : 모든 문장에 함수를 적용하고 성공한 결과만 남깁니다.
- `def uri(label):` : 공백을 밑줄로 바꾸어 이름을 URI로 만듭니다.
- `kg = Graph(); kg.bind(...)` : 저장과 SPARQL을 위한 RDF 그래프입니다.
- `nxg = nx.Graph()` : 거리로 검색할 때만 쓰는 두 번째 그래프입니다. 화살표 방향이 없으므로 양쪽으로 이동할 수 있습니다.
- `for s, p, o in triples:` : 트리플을 하나씩 처리합니다.
- `kg.add((uri(s), EX[p], uri(o)))` : 사실을 RDF에 저장합니다.
- 두 `RDFS.label` 줄: 주어와 목적어의 읽기 쉬운 이름을 저장합니다.
- `nxg.add_edge(s, o, rel=p)` : 관계 이름을 기억하면서 검색용 그래프에 연결을 추가합니다.
- `print(...)` : 개수를 보여 줍니다.
- `def retrieve(question, hops=3):` : 검색 함수입니다. `hops`는 몇 단계 떨어진 곳까지 볼지 정합니다.
- `seeds = [...]` : 이름이 질문에 나타나는 노드를 찾습니다. 이 간단한 텍스트 일치가 "엔터티 연결(entity linking)" 단계입니다. 실제 시스템은 더 똑똑한 일치를 씁니다.
- `nodes = set(seeds)` : 시드(seed)에서 시작하여 남길 노드의 집합입니다.
- `for s in seeds:`와 `nodes |= ...` 줄: 각 시드에서 `hops` 단계 안에 있는 모든 노드를 추가합니다.
- `sub = nxg.subgraph(nodes)` : 그 노드들만으로 만든 작은 그래프입니다.
- `facts` 반복문: 원래 트리플 중 양쪽 끝이 모두 작은 그래프 안에 있는 것만 골라 짧은 문장으로 씁니다. 화살표 방향을 지키려고 원래 트리플을 다시 읽습니다.
- `question = ...`와 `seeds, facts = retrieve(...)` : 질문을 하고 3단계로 검색합니다. Shakespeare에서 the UK까지가 세 단계이기 때문입니다.
- `prompt = (...)` : 지시문("use ONLY these facts"), 사실 목록, 질문을 하나의 텍스트로 합칩니다. 이것을 LLM에 보내면 그래프에 기반한 답을 얻습니다. 모델이 외부 지식을 쓰지 못하게 하려고 ONLY라는 단어를 넣습니다.
- `for i, c in enumerate(greedy_modularity_communities(nxg), 1):`와 `print(i, sorted(c))` : 그래프를 커뮤니티로 나누고, 각 그룹을 1번부터 번호를 붙여 출력합니다.

직접 해 보기: 질문을 Marlowe에 관한 질문으로 바꾸고, `hops=1`과 `hops=3`을 비교해 보세요.